omspy-api

A production-minded, multi-tenant REST API built with Django.

Authentication. Multi-tenancy. Security. One foundation for your next backend.

An API-first backend built with Django REST Framework, featuring JWT authentication, email verification, password recovery, tenant management, and PostgreSQL full-text search.

Originally developed within **SFG's internal technology stack** and open-sourced so developers can use, fork, customize, and extend the same foundation for their own applications.





Features • Quick Start • API Reference • Architecture • Testing • Security

Overview

Building a backend from scratch means repeatedly solving the same problems: authentication, account management, password recovery, permissions, and tenant isolation.

omspy-api provides a reusable foundation for those problems.

It is a versioned REST API designed for applications that need user accounts, tenant-based organization, secure authentication, and a structured backend architecture.

What You Get

- **JWT authentication** with access and refresh tokens.
- **Account security** with email verification and brute-force protection.
- **Multi-tenancy** for organizing users and application data into separate tenant contexts.
- **Email workflows** for account verification and password recovery.
- **Tenant invitations** through invitation codes.
- **PostgreSQL full-text search** for supported searchable resources.
- **Docker-based development** with Docker Compose.
- **Automated testing** with pytest and pytest-django.
- **Production-oriented deployment configuration** using Gunicorn.

Whether you're building a SaaS platform, an internal business tool, or a multi-organization application, omspy-api gives you a starting point you can extend.

> **Important:** A production-minded architecture is not the same as a security audit or a guarantee of production readiness. Review the implementation, deployment configuration, and tenant authorization rules before deploying.

✨ Features

🔐 Authentication & Account Management

- JWT access and refresh tokens.
- Token refresh and verification endpoints.
- Refresh-token revocation.
- User registration and account management.
- Password change functionality.
- Email verification workflows.
- Password reset through email.
- Brute-force protection.

🏢 Multi-Tenant Architecture

- Create and retrieve tenants.
- Update tenant information with administrator restrictions.
- List tenant members.
- Support tenant invitations.
- Look up invitation codes.
- Associate users with tenants.
- Manage tenant membership through versioned API endpoints.

Tenant isolation must be enforced server-side for every tenant-scoped operation. Authentication alone does not establish permission to access a particular tenant.

🔎 Data Access & Search

- PostgreSQL-backed persistence.
- Full-text search for supported resources.
- Django ORM for database operations.
- Django REST Framework serializers for request and response handling.
- Structured, versioned API routes.

⚙️ Deployment & Developer Experience

- Dockerfile and Docker Compose configuration.
- Gunicorn application server.
- Environment-based configuration.
- Database migrations.
- Automated tests.
- Dedicated security and contribution documentation.

🧰 Technology Stack

Layer	Technology	Purpose

Layer	Technology	Purpose
Language	Python	Backend application logic
Framework	Django 4.2	Application framework
API	Django REST Framework	REST endpoints and serialization
Authentication	Simple JWT	JWT access and refresh tokens
Database	PostgreSQL 16	Relational data and full-text search
Application server	Gunicorn	WSGI application serving
Containers	Docker	Application packaging
Orchestration	Docker Compose	Local multi-service development
Testing	pytest	Test execution
Django testing	pytest-django	Django-aware testing
Dependency management	pip	Python package installation

Compatibility

The project uses Django 4.2 LTS and PostgreSQL 16 as its documented stack.

For new production deployments in 2026, review framework support status before choosing these versions. Django 4.2's extended support period ends in April 2026, so upgrading to a currently supported Django release should be a priority. Test third-party package compatibility before upgrading.

The Python version should also match the project's dependencies, Docker image, and CI configuration.

🚀 Quick Start

The fastest way to run omspy-api is with Docker Compose.

Prerequisites

Install the following:

- Git
- Docker Engine or Docker Desktop
- Docker Compose
- A code editor

If running without Docker, you'll also need a compatible Python installation and PostgreSQL configured according to the project's settings.

1. Clone the Repository

```
git clone <your-repository-url>
cd omspy-api
```

Replace `<your-repository-url>` with the actual repository URL.

2. Configure Environment Variables

Copy the default environment template:

```
cp dot.env.default .env
```

On Windows Command Prompt:

```
copy dot.env.default .env
```

Open `.env` and configure the required values.

At minimum, verify the database connection, Django secret key, allowed hosts, email settings, and JWT-related configuration.

3. Start with Docker Compose

```
docker compose -p omspy up -d
```

If the repository specifically requires the legacy `docker-compose` executable, use:

```
docker-compose -p omspy up -d
```

The actual services and startup behavior depend on `docker-compose.yml`.

4. Apply Database Migrations

If migrations are not automatically applied by the container entrypoint, run:

```
docker compose -p omspy exec <django-service> python manage.py migrate
```

Replace `<django-service>` with the actual Django service name from your Compose configuration.

5. Verify the Application

Inspect the running containers:

```
docker compose -p omspy ps
```

Review the application logs:

```
docker compose -p omspy logs -f
```

Use the host and port configured in Docker Compose to access the API.

You're ready to start testing the endpoints.

Running Without Docker

Prefer a local Python environment? Follow these steps.

1. Create a Virtual Environment

```
python -m venv .venv
```

Activate it on Linux or macOS:

```
source .venv/bin/activate
```

On Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

2. Install Dependencies

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

3. Configure Environment Variables

Copy the environment template and fill in the required settings:

```
cp dot.env.default .env
```

Ensure PostgreSQL is running and the database credentials match your configuration.

4. Apply Migrations

```
python manage.py migrate
```

5. Start the Development Server

```
python manage.py runserver
```

The default Django development server is usually available at:

```
http://127.0.0.1:8000/
```

The API endpoints are available under the `/api/v1/` prefix, subject to the configured URL routes.

> Do not use Django's development server as the production application server. Use the deployment configuration and an appropriate production server instead.

🔑 API Reference

The API is versioned under:

```
/api/v1/
```

The following endpoints describe the documented API surface. Request formats, response schemas, permissions, and exact route behavior should be verified against the implementation.

Authentication & Tokens

Method	Endpoint	Auth	Description
`POST`	`/api/v1/token/`	No	Obtain access and refresh tokens
`POST`	`/api/v1/token/refresh/`	No*	Refresh an access token
`POST`	`/api/v1/token/verify/`	No*	Verify token validity
`POST`	`/api/v1/token/revoke/`	Yes*	Revoke a refresh token

*Standard Simple JWT refresh and verification endpoints commonly use the token supplied in the request body rather than requiring an existing access token. Refresh-token revocation typically requires a configured blacklist implementation. Verify the actual project configuration.

User Management

Method	Endpoint	Auth	Description
POST	/api/v1/users/	No	Create an account
GET	/api/v1/users/<id>/	Yes	Retrieve a user
PUT	/api/v1/users/<id>/	Yes	Update a user
DELETE	/api/v1/users/<id>/	Yes	Delete a user
PUT	/api/v1/users/<id>/password/	Yes	Change a password
GET	/api/v1/users/<id>/tenants/	Yes	List a user's tenants

Email Verification & Password Recovery

Method	Endpoint	Auth	Description
POST	/api/v1/email/signup/verification/	No	Request an account verification email
POST	/api/v1/password/reset-code/	No	Request a password reset email
POST	/api/v1/password/reset/	No	Reset a password

Public authentication-related endpoints should have appropriate rate limits and abuse protection.

Tenant Management

Method	Endpoint	Auth	Description
POST	/api/v1/tenants/	Yes	Create a tenant
GET	/api/v1/tenants/<domain>/	Yes	Retrieve a tenant
PUT	/api/v1/tenants/<domain>/	Yes	Update a tenant; administrator only
GET	/api/v1/tenants/<domain>/users/	Yes	List tenant members
POST	/api/v1/tenants/<domain>/users/	Yes	Join a tenant through an invitation
GET	/api/v1/tenants/<domain>/users/<id>/	Yes	Retrieve a tenant member
POST	/api/v1/tenants/<domain>/invitation-codes/	Yes	Send an invitation; administrator only

Method	Endpoint	Auth	Description
POST	/api/v1/tenants/invited/	Yes	Look up an invitation

Authentication Headers

For protected endpoints, send the access token using the standard Bearer scheme:

```
Authorization: Bearer <access-token>
```

Include JSON headers where appropriate:

```
Content-Type: application/json
Accept: application/json
```

Example: Obtain a Token

```
curl -X POST http://127.0.0.1:8000/api/v1/token/ \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -d '{
    "email": "developer@example.com",
    "password": "your-password"
  }'
```

The request fields must match the configured authentication serializer. Some Simple JWT configurations expect `username` rather than `email`.

Example: Retrieve a User

```
curl http://127.0.0.1:8000/api/v1/users/1/ \
  -H "Authorization: Bearer <access-token>" \
  -H "Accept: application/json"
```

A valid token does not automatically grant access to every user record. Object-level permissions should determine whether the authenticated user may retrieve the requested resource.

Architecture

omspy-api separates API routing, serialization, application models, and business-facing views into dedicated modules.

```
omspy-api/
│
├── api/
│   ├── common/             # Shared constants, exceptions, utilities
│   ├── resources/
│   │   └── v1/             # Versioned API views
│   ├── serializers/        # DRF serializers
│   ├── tests/              # API test suite
│   └── urls/               # API URL configuration
│
├── core/
│   ├── admin/              # Django admin configuration
│   ├── forms/              # Forms
│   ├── management/
│   │   └── commands/       # Custom management commands
│   ├── migrations/         # Database migrations
│   ├── models/             # Application models
│   └── views/              # Application views
│
├── config/
│   ├── settings.py         # Application settings
│   ├── urls.py             # Root URL configuration
│   ├── wsgi.py             # WSGI entry point
│   └── asgi.py             # ASGI entry point
│
├── Dockerfile
├── docker-compose.yml
├── docker-compose.test.yml
├── gunicorn.conf.py
├── requirements.txt
├── manage.py
└── README.md
```

This is a logical representation of the documented structure. The actual repository may contain additional files or use a different module layout.

Why This Structure?

- **Versioned resources:** API changes can be organized by version.
- **Dedicated serializers:** Request validation and response serialization remain centralized.
- **Shared utilities:** Common behavior can be reused across endpoints.
- **Django models:** Database entities and relationships are represented through the ORM.
- **Custom management commands:** Repeatable operational tasks can run through Django's command interface.
- **Automated tests:** API behavior can be tested independently of manual requests.

🧪 Testing

Testing is handled with pytest and pytest-django.

Run the Test Suite

If the database and other dependencies are already available:

```
pytest
```

Run Tests Through Docker Compose

Start the test environment:

```
docker compose -p omspy-test -f docker-compose.test.yml up -d
```

Then execute pytest in the appropriate test container or configured environment.

For example, if the test service is named `test`:

```
docker compose -p omspy-test -f docker-compose.test.yml exec test pytest
```

Adjust the service name to match `docker-compose.test.yml`.

What Should Be Tested?

Area	Important Test Cases
Authentication	Valid login, invalid credentials, refresh, revocation
Users	Registration, profile updates, password changes
Email	Verification and password reset flows
Tenants	Creation, membership, invitation handling
Authorization	Unauthorized access and privilege escalation
Multi-tenancy	Cross-tenant data isolation
Search	Expected full-text search results
Security	Rate limits and brute-force protection

The test suite should verify not only that authorized requests succeed, but also that unauthorized and cross-tenant requests fail.

⚙ Environment Configuration

The environment template is:

```
dot.env.default
```

Copy it to `.env` and configure the variables documented in that file.

Common configuration categories include:

Category	Purpose
Django secret key	Cryptographic application secret
Debug mode	Development versus production behavior
Allowed hosts	Permitted hostnames
Database	PostgreSQL connection details
Email	Verification and password reset delivery
JWT	Token lifetime and authentication behavior
Security	HTTPS, cookies, trusted origins, and related settings
Docker	Service configuration and port mapping

The actual environment variable names depend on `config/settings.py` and the project's configuration conventions.

Never commit production secrets or a populated `.env` file.

Docker & Deployment

Docker provides a repeatable environment for developing and running the API.

The repository includes:

- `Dockerfile` — application image configuration.
- `docker-compose.yml` — primary service orchestration.
- `docker-compose.test.yml` — test environment orchestration.
- `gunicorn.conf.py` — Gunicorn configuration.

Recommended Production Practices

- Use supported Django and Python versions.
- Disable Django debug mode.
- Configure `ALLOWED_HOSTS` explicitly.
- Store secrets outside the source repository.
- Serve the API over HTTPS.
- Run Django's deployment checks.

- Apply database migrations through a controlled release process.
- Configure logging and error monitoring.
- Add database backups and test recovery procedures.
- Monitor email delivery and background tasks, where applicable.
- Run the application with a production server rather than `runserver`.

Before deployment, run:

```
python manage.py check --deploy
```

Resolve relevant warnings for your deployment environment.

Security

Security is a core concern for an API handling accounts and tenant membership.

The project documents JWT authentication, email verification, and brute-force protection. Their exact guarantees depend on the implementation and deployment configuration.

Security Principles

Authentication

Use appropriate token lifetimes and protect refresh-token workflows.

Authorization

Apply permission checks to every protected operation, including individual user records and tenant resources.

Tenant isolation

Validate tenant membership and permissions server-side. Never trust a tenant domain or tenant identifier supplied by the client without checking access.

Password security

Use Django's password hashing framework. Never store plaintext passwords or expose sensitive authentication details.

Brute-force protection

Apply rate limits to login, registration, password recovery, and verification endpoints.

Secret management

Keep Django secret keys, database passwords, email credentials, and other secrets outside version control.

Transport security

Use HTTPS in production and configure secure cookies and trusted origins where applicable.

Dependency management

Regularly review Django, Django REST Framework, Simple JWT, and other installed packages for security updates.

Security Reporting

See `SECURITY.md` for the project's security reporting process.

Please report vulnerabilities responsibly rather than publishing exploitable details before maintainers can investigate.

🗺 Roadmap

Potential improvements for future versions:

- ☐ Upgrade to a currently supported Django release.
- ☐ Publish OpenAPI documentation.
- ☐ Add continuous integration with GitHub Actions.
- ☐ Expand automated authorization tests.
- ☐ Add tenant-isolation integration tests.
- ☐ Improve structured logging and observability.
- ☐ Add API throttling documentation and configuration examples.
- ☐ Add container health checks.
- ☐ Document production deployment and backup procedures.
- ☐ Expand developer onboarding documentation.

This list represents possible future work, not a claim that these capabilities are already implemented.

🤝 Contributing

omspy-api is open source. Contributions, bug reports, and improvements are welcome.

Before contributing:

1. Read `CONTRIBUTING.md`.
2. Check existing issues and discussions.
3. Create a focused branch for your changes.
4. Add tests for new behavior.
5. Run the test suite.
6. Document relevant API or configuration changes.
7. Submit a pull request describing the problem and solution.

Security-sensitive changes should include tests for unauthorized access and tenant isolation.

License

This project is licensed under the MIT License.

Copyright © SFG

See `LICENSE` for the complete license text.

Built at SFG

SFG is a technology studio that builds software, AI systems, automations, and digital products for businesses.

omspy-api is part of our effort to make useful backend infrastructure available to the wider developer community.

We build technology for real business problems, and we believe useful foundations should be reusable.

Website: seofg.xyz
GitHub: github.com/reabot6

Build on it. Fork it. Make it yours.

omspy-api — A reusable foundation for your next Django backend.

Built by Reabot6 · Open-sourced by SFG